TEK4090

Introduction to Modern Control

Lecture 6: Feedforward Control

Mathias Hudoba de Badyn

October 9, 2025

Back to Challenge 2:

- observer dynamics → $\dot{\hat{x}} = A\hat{x} + Bu + L(y - C\hat{x})$

We saw optimal gains for control, what about observers?

How to choose the observer gain optimally?

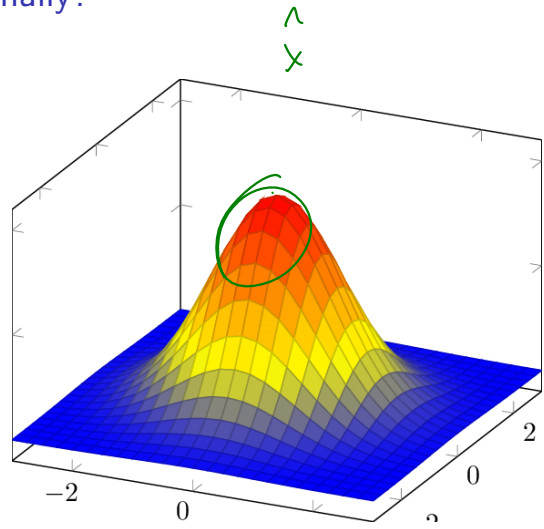
In general, measurements are **noisy**.

$$x_{k+1} = Ax_k + Bu_k + Fv_k$$
$$y_k = Cx_k + Du_k + w_k$$

$$E[v_k] = 0, \; E[w_k] = 0$$

$$E[v_k v_{k+1}^T] = \begin{cases} 0 & k \neq j \\ Q_k & k = j \end{cases}$$

$$E[w_k w_{k+1}^T] = \begin{cases} 0 & k \neq j \\ R_k & k = j \end{cases}$$

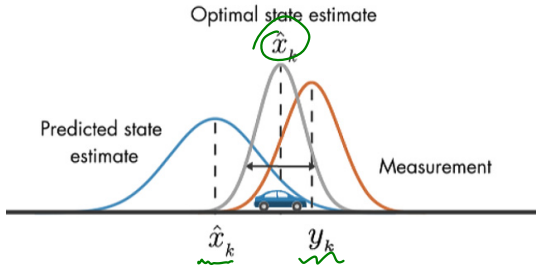


How to choose the observer gain?

With noisy measurements, the estimate is now a **random variable**.

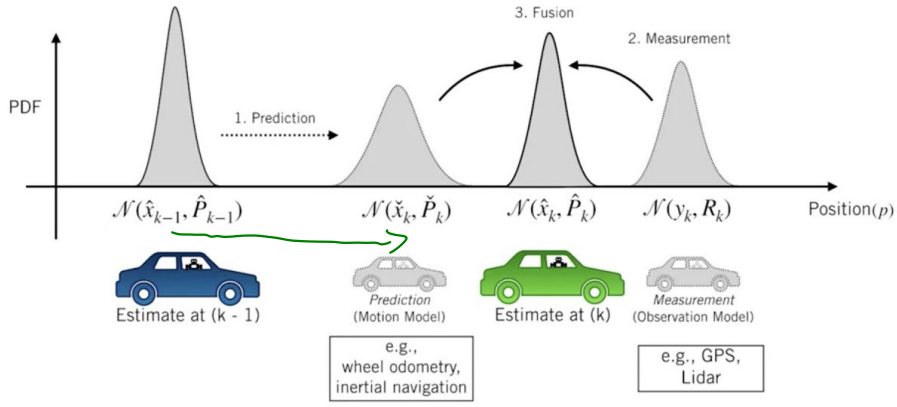
If L stabilizes the observer dynamics, then

$$E[e_k] = E[\hat{x}_k - x_k] \to 0.$$



Instead, we can optimize the 'width' (or, covariance) of the estimate

Optimal Observers



Kalman Filter Derivation

Truth model $x_{k+1} = Ax_k + Bu_k + Fw_k$ $\quad e_k = \hat{x}_k - x_k$

$\tilde{y}_k = Cx_k + v_k$

$E[v_k w_k^T] = 0$, $E[w_k w_j^T] = \begin{cases} 0 & k \neq j \\ Q_k & k = j \end{cases}$ $\quad E[v_k v_j^T] = \begin{cases} 0 & k \neq j \\ R_k & k = j \end{cases}$

at every timestep k, 2 things happen

propagate: $\hat{x}_{k+1}^- = A\hat{x}_k^+ + Bu_k$

update: $\hat{x}_k^+ = \hat{x}_k^- + K_k[\tilde{y}_k - C\hat{x}_k^-]$

+: updated w/ measurement

-: not updated

error covariance $P_k^\pm = E[e_k^\pm e_k^{\pm T}]$, $e_k^\pm = \hat{x}_k^\pm - x_k$

$e_{k+1}^- = Ae_k^+ - Fw_k$

Kalman Filter Derivation

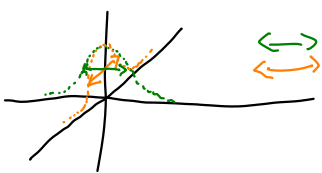
① $P_{k+1}^- = A P_k^+ A^T + F Q_k F^T$ → propagation of $P_k^+ \mapsto P_{k+1}^-$

For the update, we want P_k^+ in terms of K_k

② $\hat{x}_k^+ = (I - K_k C)\hat{x}_k^- + K_k C x_k + K_k v_k$

③ $e_k^+ = (I - K_k C) e_k^- + K_k v_k$

④ $P_k^+ = (I - K_k C) \underbrace{E[e_k^- e_k^{-T}]}_{P_k^-} (I - K_k C) + K_k R_k K_k$



$\Longleftrightarrow$ } widths are related to evals of P_k^+

↳ minimize trace of P_k^+

Kalman Filter Derivation

$\min_{K_k} \; Tr(P_k^+)$ s.t. eqn (4) 5 → take derivative with respect to P_k^+ (Matrix cookbook)

$$\frac{\partial}{\partial K_n} Tr(P_n^+) = -2(I - K_n C) P_n^- C^T + 2 K_n R_n = 0$$

$$K_n (C P_n^- C^T + R) = P_n^- C^T$$

$$\boxed{K_n = P_n^- C^T (C P_n^- C^T + R)^{-1}}$$

Kalman Filter Derivation

Kalman Filter Derivation

Kalman Filter Derivation

Kalman Filter Derivation

Kalman Filter

Model: $x_{k+1} = Ax_k + Bu_k + Fv_k$

$y_k = Cx_k + Du_k + w_k$

Kalman Filter

Model: $x_{k+1} = Ax_k + Bu_k + Fv_k$
$y_k = Cx_k + Du_k + w_k$

Initialize: $\hat{x}_0 = x_0, \; P_0 = P$

Kalman Filter

Model: $x_{k+1} = Ax_k + Bu_k + Fv_k$
$y_k = Cx_k + Du_k + w_k$

Initialize: $\hat{x}_0 = x_0, \; P_0 = P$

Gain: $K_k = P_k^- C^T \left(CP_k^- C^T + R_k \right)^{-1}$

Kalman Filter

Model:
$$x_{k+1} = Ax_k + Bu_k + Fv_k$$
$$y_k = Cx_k + Du_k + w_k$$

Initialize:
$$\hat{x}_0 = x_0, \; P_0 = P$$

Gain:
$$K_k = P_k^- C^T \left(CP_k^- C^T + R_k \right)^{-1}$$

Update:
$$\hat{x}_k^+ = \hat{x}_k^- + K_k \left(\tilde{y}_k - C\hat{x}_k^- \right)$$
$$P_k^+ = \left(I - K_k C \right) P_k^-$$

Kalman Filter

Model:
$$x_{k+1} = Ax_k + Bu_k + Fv_k$$
$$y_k = Cx_k + Du_k + w_k$$

Initialize:
$$\hat{x}_0 = x_0, \; P_0 = P$$

Gain:
$$K_k = P_k^- C^T \left(CP_k^- C^T + R_k\right)^{-1}$$

Update:
$$\hat{x}_k^+ = \hat{x}_k^- + K_k \left(\tilde{y}_k - C\hat{x}_k^-\right)$$
$$P_k^+ = \left(I - K_k C\right) P_k^-$$

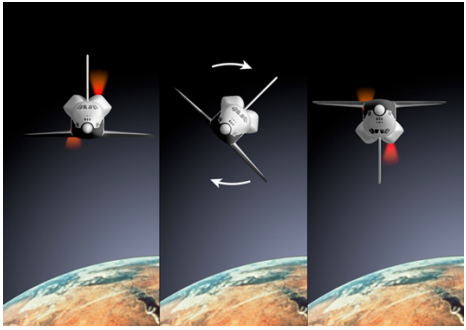
Propagate:
$$\hat{x}_{k+1}^- = A\hat{x}_k^+ + Bu_k$$
$$P_{k+1}^- = AP_k^+ A^T + BQ_k B^T$$

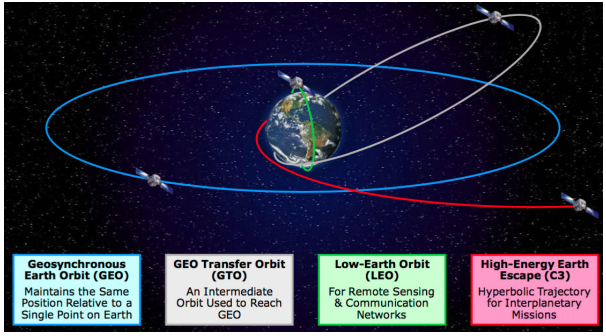
k+1 ↦ k

Guidance, Navigation, & Control

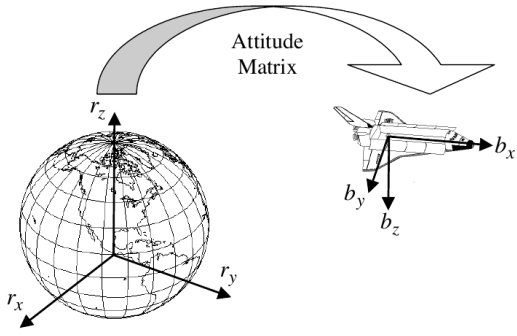
Spacecraft GNC

Spacecraft GNC





Attitude



Reference and body frames:

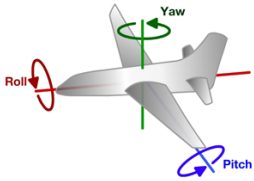
$$b = b_x\hat{b}_1 + b_y\hat{b}_2 + b_z\hat{b}_3$$
$$r = r_x\hat{r}_1 + r_y\hat{r}_2 + r_z\hat{r}_3$$

Transformation between the two:

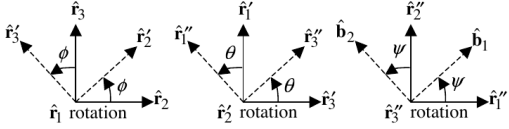
$$\begin{bmatrix}\hat{b}_1\\ \hat{b}_2\\ \hat{b}_3\end{bmatrix} = A\begin{bmatrix}\hat{r}_1\\ \hat{r}_2\\ \hat{r}_3\end{bmatrix} \quad (1)$$

Euler Angles

One can represent a rotation via the **Euler angles:** roll ϕ, pitch θ, and yaw ψ.

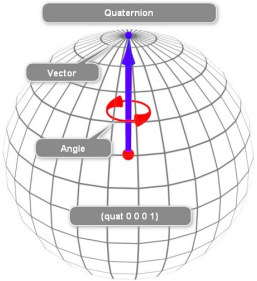


$$r' = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\phi & \sin\phi \\ 0 & -\sin\phi & \cos\phi \end{bmatrix} r \qquad r'' = \begin{bmatrix} \cos\theta & 0 & -\sin\theta \\ 0 & 1 & 0 \\ \sin\theta & 0 & \cos\theta \end{bmatrix} r' \qquad b = \begin{bmatrix} \cos\psi & \sin\psi & 0 \\ -\sin\psi & \cos\psi & 0 \\ 0 & 0 & 1 \end{bmatrix} r''$$



Quaternions

The Euler angles have a singularity when the attitude is pointed at a pole, so spacecraft attitude is represented as a **quaternion** instead.



$$q = \begin{bmatrix} \rho \\ q_4 \end{bmatrix} \in \mathbb{R}^4$$

$$q^T q = 1$$

$$A(q) = \Xi^T(q)\Psi(q)$$

$$\Xi(q) = \begin{bmatrix} q_4 I_{3\times 3} + [\rho\times] \\ -\rho^T \end{bmatrix}$$

$$\Psi(q) = \begin{bmatrix} q_4 I_{3\times 3} - [\rho\times] \\ -\rho^T \end{bmatrix}$$

$$[\rho\times] = \begin{bmatrix} 0 & -\rho_3 & \rho_2 \\ \rho_3 & 0 & -\rho_1 \\ -\rho_2 & \rho_1 & 0 \end{bmatrix}$$

matrix multiplication of $[\rho\times]$ is equivalent to cross product

Rigid Body Dynamics

Dynamics: Euler Angles

Kinematics: Quaternions

- H: Angular momentum

$$\dot{H} = J\dot{\omega}$$
$$= -[\omega\times]J\omega + L$$

$$\dot{q} = \frac{1}{2}\Omega(\omega)q$$
$$\Omega(\omega) = \begin{bmatrix} -[\omega\times] & \omega \\ -\omega^T & 0 \end{bmatrix}$$

The torque L comes from **thrusters**, **reaction wheels** or **magnetorquers**.

Rigid Body Dynamics

- H: Angular momentum
- J: Moment of Inertia

Dynamics: Euler Angles

$$\begin{aligned}\dot{H} &= J\dot{\omega} \\ &= -[\omega\times]J\omega + L\end{aligned}$$

Kinematics: Quaternions

$$\dot{q} = \frac{1}{2}\Omega(\omega)q$$

$$\Omega(\omega) = \begin{bmatrix} -[\omega\times] & \omega \\ -\omega^T & 0 \end{bmatrix}$$

The torque L comes from **thrusters**, **reaction wheels** or **magnetorquers**.

Rigid Body Dynamics

- H: Angular momentum
- J: Moment of Inertia
- L: Applied torque

Dynamics: Euler Angles

$$\begin{aligned}\dot{H} &= J\dot{\omega} \\ &= -[\omega\times]J\omega + L\end{aligned}$$

Kinematics: Quaternions

$$\dot{q} = \frac{1}{2}\Omega(\omega)q$$

$$\Omega(\omega) = \begin{bmatrix} -[\omega\times] & \omega \\ -\omega^T & 0 \end{bmatrix}$$

The torque L comes from **thrusters**, **reaction wheels** or **magnetorquers**.

Rigid Body Dynamics

- H: Angular momentum
- J: Moment of Inertia
- L: Applied torque
- ω: Angular velocity

Dynamics: Euler Angles

$$\begin{aligned}\dot{H} &= J\dot{\omega} \\ &= -[\omega\times]J\omega + L\end{aligned}$$

Kinematics: Quaternions

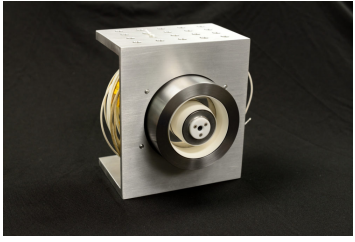
$$\dot{q} = \frac{1}{2}\Omega(\omega)q$$

$$\Omega(\omega) = \begin{bmatrix} -[\omega\times] & \omega \\ -\omega^T & 0 \end{bmatrix}$$

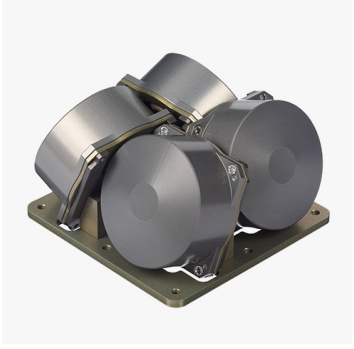
The torque L comes from **thrusters**, **reaction wheels** or **magnetorquers**.

Spacecraft Actuators

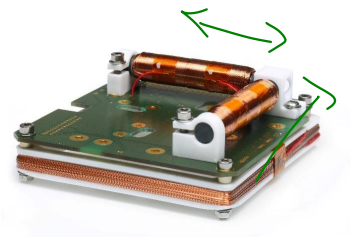
Thrusters



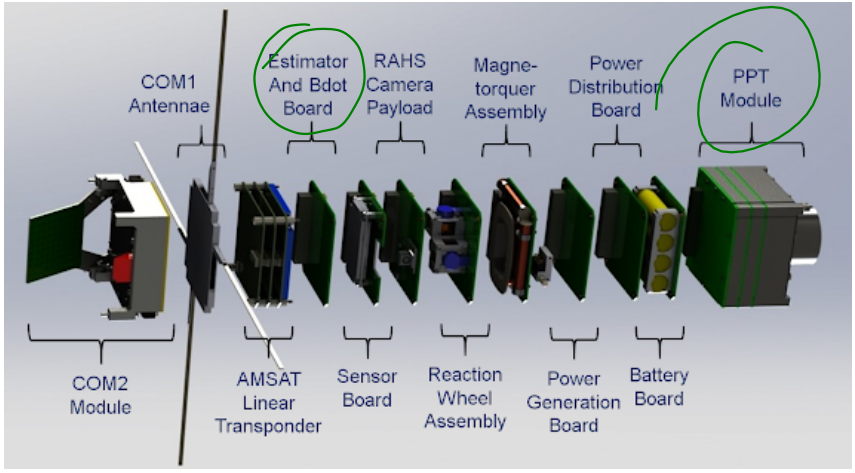
reaction wheels



Magnetorquers



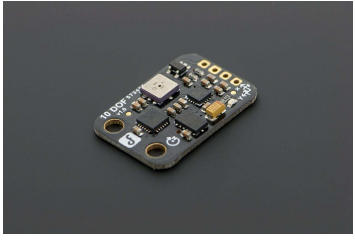
Plasma Thruster



Measurement

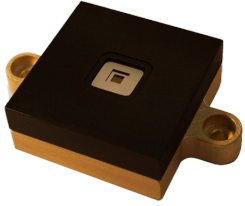
have a bias!

Inertial Measurement unit



Star tracker

Sun sensor

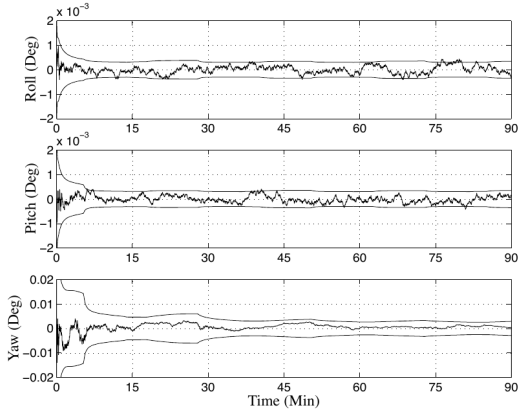


Quaternion Extended Kalman Filter

quatodynamics → nonlinear systems

Initialize	$\hat{\mathbf{q}}(t_0) = \hat{\mathbf{q}}_0, \quad \hat{\beta}(t_0) = \hat{\beta}_0$ $P(t_0) = P_0$
Gain	$K_k = P_k^- H_k^T(\hat{\mathbf{x}}_k^-)[H_k(\hat{\mathbf{x}}_k^-)P_k^- H_k^T(\hat{\mathbf{x}}_k^-) + R]^{-1}$ $H_k(\hat{\mathbf{x}}_k^-) = \begin{bmatrix} [A(\hat{\mathbf{q}}_k^-)\mathbf{r}_1\times] & 0_{3\times3} \\ \vdots & \vdots \\ [A(\hat{\mathbf{q}}_k^-)\mathbf{r}_n\times] & 0_{3\times3} \end{bmatrix}\Big\vert_{t_k}$
Update	$P_k^+ = [I - K_k H_k(\hat{\mathbf{x}}_k^-)]P_k^-$ $\Delta\hat{\mathbf{x}}_k^+ = K_k[\bar{\mathbf{y}}_k - \mathbf{h}_k(\hat{\mathbf{x}}_k^-)]$ $\Delta\hat{\mathbf{x}}_k^+ \equiv [\delta\hat{\alpha}_k^{+T} \; \Delta\hat{\beta}_k^{+T}]^T$ $\mathbf{h}_k(\hat{\mathbf{x}}_k^-) = \begin{bmatrix} A(\hat{\mathbf{q}}_k^-)\mathbf{r}_1 \\ A(\hat{\mathbf{q}}_k^-)\mathbf{r}_2 \\ \vdots \\ A(\hat{\mathbf{q}}_k^-)\mathbf{r}_n \end{bmatrix}\Big\vert_{t_k}$ $\hat{\mathbf{q}}_k^+ = \hat{\mathbf{q}}_k^- + \frac{1}{2}\Xi(\hat{\mathbf{q}}_k^-)\delta\hat{\alpha}_k^+, \quad$ re-normalize quaternion $\hat{\beta}_k^+ = \hat{\beta}_k^- + \Delta\hat{\beta}_k^+$
Propagation	$\hat{\omega}(t) = \omega(t) - \hat{\beta}(t)$ $\dot{\hat{\mathbf{q}}}(t) = \frac{1}{2}\Xi(\hat{\mathbf{q}}(t))\,\hat{\omega}(t)$ $\dot{P}(t) = F(t)P(t) + P(t)F^T(t) + G(t)Q(t)G^T(t)$ $F(t) = \begin{bmatrix} -[\hat{\omega}(t)\times] & -I_{3\times3} \\ 0_{3\times3} & 0_{3\times3} \end{bmatrix}, \quad G(t) = \begin{bmatrix} -I_{3\times3} & 0_{3\times3} \\ 0_{3\times3} & I_{3\times3} \end{bmatrix}$

bias



Matlab - simulating ODEs


```m
clear all; close all; clc;

% simulation timespan
dt=0.1;
t_space = [0:dt:10];

% initial condition
x_0 = [10;10];

% sinusoidal input with zero-order-hold
u = cos(t_space);
x_array = zeros(2,length(t_space));

x_array(:,1) = x_0;

for ii = 1:length(t_space)-1

    t_int = [t_space(ii), t_space(ii+1)];

    %simulate ODE with input
    [t_out,y_out] = ode45(@fn_rhs,t_int,x_array(:,ii),[],u(ii));
    x_array(:,ii+1) = y_out(end,:)';

end

 plot(t_space, x_array(1,:)), hold on
 plot(t_space, x_array(2,:))
```

ODE solvers

parameters for solver

pass u(ii) as a constant

initial cond

time domain

```m
function [x_dot] = fn_rhs(t, x, u)
%fn_rhs: a `right-hand-side function' of the ode x_dot = f(t,x,u)

x_dot = exp(-t)*[cos(x(1)); sin(x(2))] + exp(-0.1*t)*[0;1]*u;

end
```

Recap from Last Time

- Observability

Recap from Last Time

- Observability
- Estimators & State Estimation

Recap from Last Time

- Observability
- Estimators & State Estimation
- Optimal State Estimation: Kalman Filtering

Recap from Last Time

Note: all code available for Crassidis & Junkins

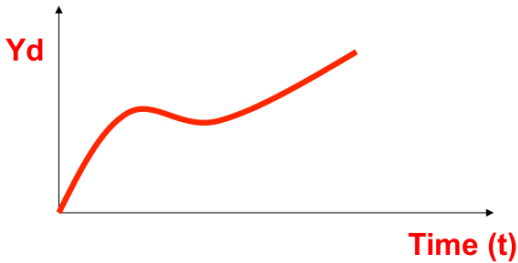
- Observability
- Estimators & State Estimation
- Optimal State Estimation: Kalman Filtering
- Applications to Spacecraft GNC

Reading Assignment

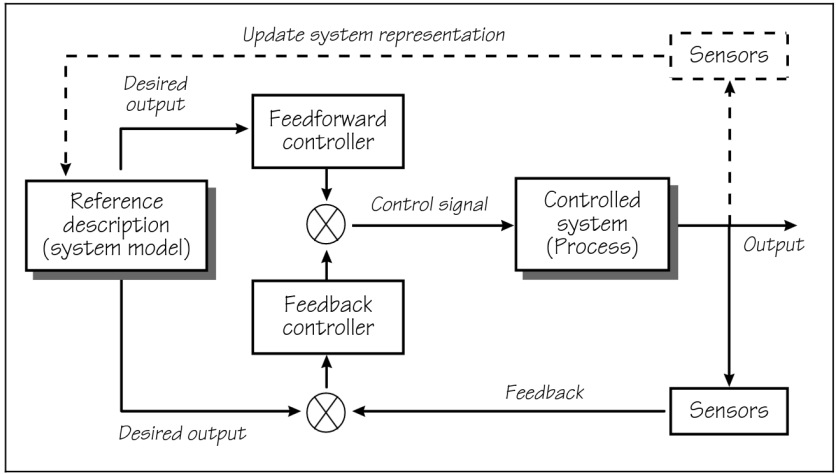
1. Notes by Santosh Devasia (University of Washington)

Motivation

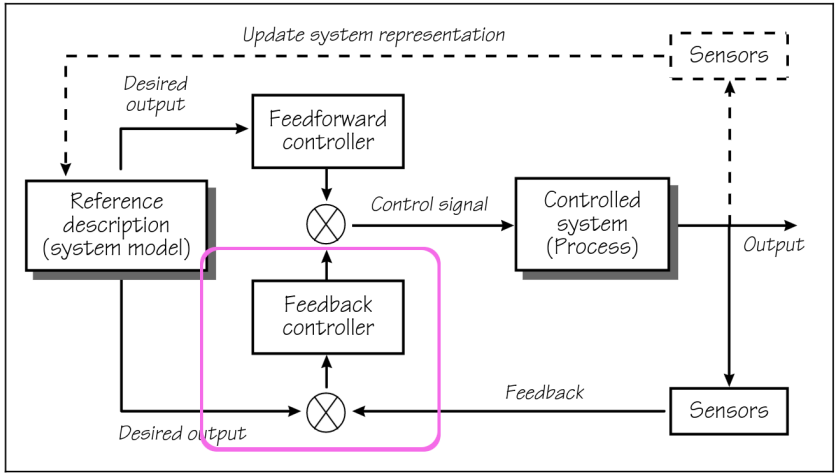
Goal: Find the input $u(t)$ that achieves a desired output trajectory $y(t)$



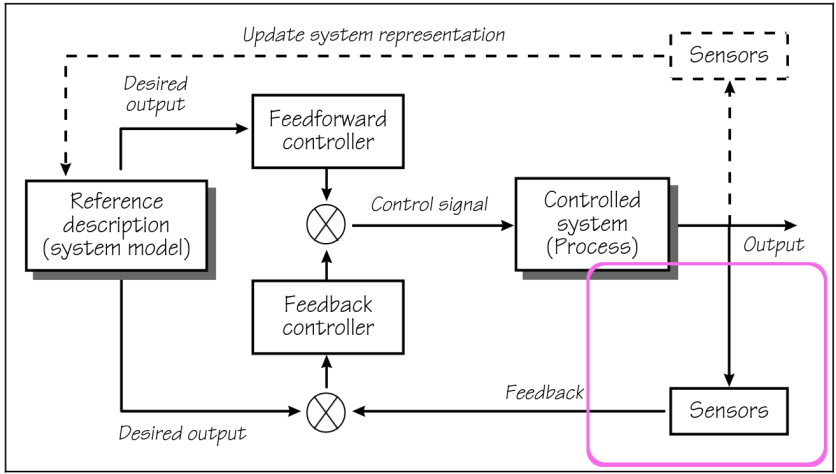
Control Block Diagram



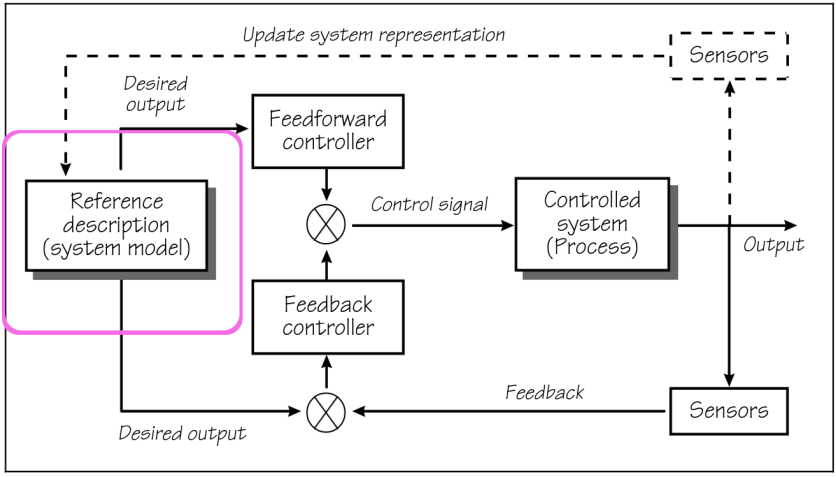
Control Block Diagram – Feedback



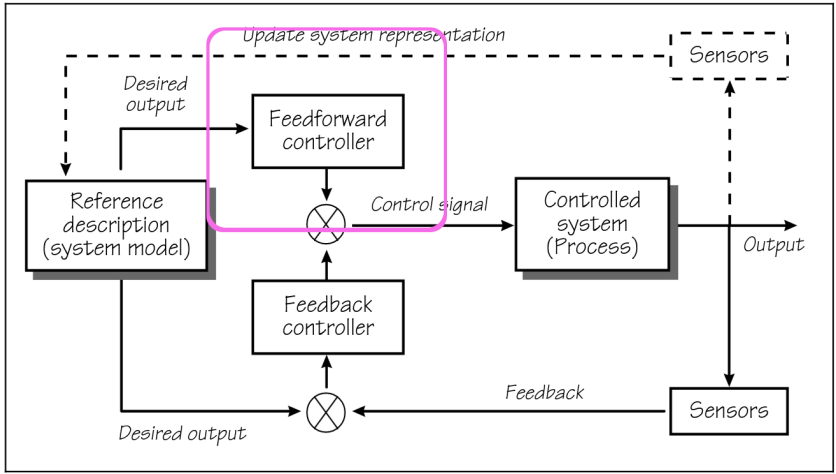
Control Block Diagram – Estimation



Control Block Diagram – System Identification



Control Block Diagram – Feedforward Control



System Inversion

$Y(s) = G(s)U(s)$ — calculate $Y(s)$, given $U(s)$

given $Y(s)$ $\{y(t)\}$ — how do we get $U(s)$ $\{u(t)\}$

Basic Example: Minimum-Phase SISO system

Transfer Function

$$\frac{Y(s)}{U(s)} := G(s) = \frac{(s+1)}{(s+2)}$$

$$\implies Y(s) = \frac{(s+2)}{(s+1)}U(s)$$

System Inversion

Basic Example: Minimum-Phase SISO system

Transfer Function

$$\frac{Y(s)}{U(s)} := G(s) = \frac{(s-1)}{(s-2)}$$

$$\implies Y(s) = \frac{(s-1)}{(s-2)} U(s)$$

Inverse Transfer Function

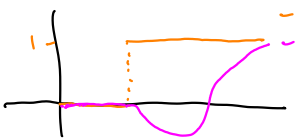
reciprocal

$$\frac{U(s)}{Y(s)} := G^{-1}(s) = \frac{(s-1)}{(s-2)}$$

$$\implies U(s) = \frac{(s-2)}{(s-1)} Y(s)$$

System Inversion

non-minimum phase: zeros of the TF (poles of inv TF) are positive



Basic Example: Minimum-Phase SISO system

Transfer Function

$$\frac{Y(s)}{U(s)} := G(s) = \frac{(s+1)}{(s+2)}$$
$$\implies Y(s) = \frac{(s+2)}{(s+1)}U(s)$$

Inverse Transfer Function

$$\frac{U(s)}{Y(s)} := G^{-1}(s) = \frac{(s+1)}{(s+2)}$$
$$\implies U(s) = \frac{(s+2)}{(s+1)}Y(s)$$

Note that both $G(s)$ and G^{-1} are **open-loop stable**

Setup

Linear SISO System

$$\dot{x} = Ax + Bu$$
$$y = Cx$$
$$u, y \in \mathbb{R}$$

Transfer Function

$$G(s) = C(sI - A)^{-1}B$$

Setup

"weaker" observability condition (controlability)

Assumption: Well-defined relative degree.

The system has a well-defined relative degree r if,

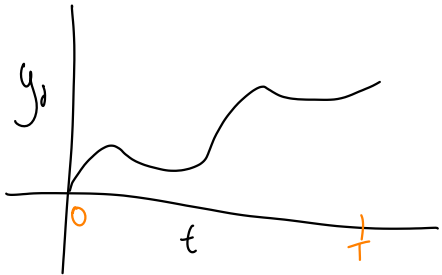
$$CA^j B = 0 \quad j < r-1$$
$$CA^{r-1}B \neq 0$$

scalar, $\Rightarrow$ invertible

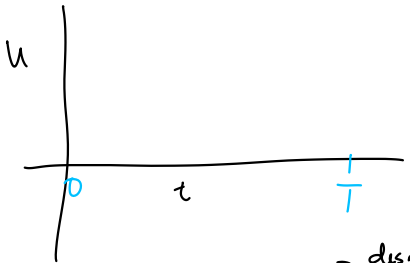
Setup

The relative degree r corresponds to the difference between the order of the denominator and the numerator of the transfer function

The Plan

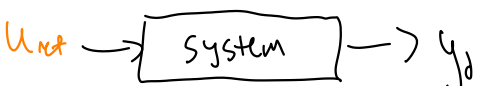


"inverse system"



$$\dot{x} = Ax + B u_{ref}$$

$$y = Cx = y_d(t)$$



$$\dot{\eta} = A_{inv}\eta + B_{inv} y_d$$

desired trajectory

$$u_{ref} = C_{inv}\eta + D_{inv} y_d$$

↳ output

Step 1: Differentiate Until the Input Appears

$$y = Cx$$

$$\dot{y} = C\dot{x} = C(Ax + Bu) = CAx + CBu$$

($CBu \to 0$)

$$\ddot{y} = CA\dot{x} = CA^2x + CABu$$

($CABu \to 0$)

$$\vdots$$

$$\frac{d^r}{dt^r}y = CA^rx + CA^{r-1}Bu(t)$$

($CA^{r-1}B \neq 0$ by assumption)

$$y^{(r)} = CA^rx + CA^{r-1}Bu$$

$CA^jB = 0$ if $j < r-1$

Step 1: Differentiate Until the Output Appears

$$y^{(r)} = \underbrace{CA^r x}_{A_y} + \underbrace{CA^{r-1}B}_{B_y}\overset{\neq 0}{u} = A_y x(t) + B_y u(t)$$

B_y → invertible

$$u_{inv} = B_y^{-1}\left(y^{(r)} - A_y x(t)\right)$$

problem: I don't know $x(t)$ (we know $y(t)$)

so, we can't easily get $x(t)$ from $y(t)$

Step 2: Decouple the State into Known/Unknown Parts

"known" parts

$$\begin{bmatrix} y \\ \dot{y} \\ \vdots \\ y^{(r-1)} \end{bmatrix} = \begin{bmatrix} C \\ CA \\ \vdots \\ CA^{r-1} \end{bmatrix} x$$

full row rank, get x from y

$$z(t) = \begin{bmatrix} y_d \\ \dot{y}_d \\ \vdots \\ y^{(r-1)} \\ \hline \eta \end{bmatrix} := \begin{bmatrix} \eta_y \\ \hline \eta \end{bmatrix} = \underbrace{\begin{bmatrix} C \\ CA \\ \vdots \\ CA^{r-1} \\ \hline T_\eta \end{bmatrix}}_{T} x = \begin{bmatrix} T_y \\ \hline T_\eta \end{bmatrix} x(t)$$

"unknown"

T_η is defined arbitrarily, so T is invertible

Step 2: Decouple the State into Known/Unknown Parts

$$x_{ref} = T^{-1} \begin{bmatrix} y_d \\ \eta_{ref} \end{bmatrix} \rightsquigarrow \eta(t),$$ so $x(t)$ does what it needs to do, so that $y = Cx = y_d$

$$= \underbrace{\left[T_\ell^{-1} \mid T_r^{-1} \right]}_{\text{notational abuse}} \begin{bmatrix} y_d \\ \eta_{ref} \end{bmatrix} = T_\ell^{-1} y_d + T_r^{-1} \eta_{ref}$$

next: plug into u_{inv} to get system of eqns for η_{ref}

Step 3: Write the Inverse Input Dynamics

$$u_{inv} = B_y^{-1}\left(y_d^{(r)} - A_y x_{ref}\right)$$

$$= B_y^{-1}\left(y_d^{(r)} - A_y\left(T_\ell^{-1} y_d + T_r^{-1}\eta_{ref}\right)\right)$$

$$= \underbrace{B_y^{-1} y_d^{(r)} - B_y^{-1} A_y T_\ell^{-1} y_d}_{\text{known}} - \underbrace{B_y^{-1} A_y T_r^{-1}\eta_{ref}}_{\text{unknown}}$$

$$= \underbrace{-B_y^{-1} A_y T_r^{-1}\eta_{ref}}_{} + \underbrace{\left[-B_y^{-1} A_y T_\ell^{-1} \mid B_y^{-1}\right]}_{D_{inv}} \underbrace{\begin{bmatrix} y_d \\ \hline y_d^{(r)} \end{bmatrix}}_{a_{y_d}}$$

Step 3: Write the Inverse Input Dynamics

$$u_{inv} = C_{inv}\eta(t) + D_{inv}y_d$$

$$z = \begin{bmatrix} y_d \\ \eta \end{bmatrix} = Tx \hookrightarrow \begin{bmatrix} T_y \\ T_\eta \end{bmatrix} x$$

Recover η dynamics from x

$\eta = T_\eta x$

short & fat matrix

Step 3: Write the Inverse Input Dynamics

$$\eta = T_\eta\, x$$

$$\dot\eta = T_\eta \dot x = T_\eta (Ax + Bu_{inv})$$

$$x_{ref} = T^{-1}\begin{bmatrix} y_d \\ \eta_{ref}\end{bmatrix} = \begin{bmatrix} T_\ell^{-1} \mid T_r^{-1}\end{bmatrix}\begin{bmatrix} y_d \\ \eta_{ref}\end{bmatrix} = T_\ell^{-1} y_d + T_r^{-1}\eta_{ref}$$

$$= T_\eta \left[Ax + B\left[-B_y^{-1}A_y T_r^{-1}\right]\eta + \left[-B_y^{-1}A_y T_\ell^{-1} \mid B_y^{-1}\right] y_d \right]$$

$$= T_\eta \left[A\left(T_\ell^{-1} y_d + T_r^{-1}\eta_{ref}\right) + B\left[-B_y^{-1}A_y T_r^{-1}\right]\eta + \left[-B_y^{-1}A_y T_\ell^{-1} \mid B_y^{-1}\right]\begin{bmatrix} y_d \\ \dot y_d \end{bmatrix}\right]$$

$$= T_\eta \left[AT_r^{-1} - BB_y^{-1}A_y T_r^{-1}\right]\eta + T_\eta \left[AT_\ell^{-1} - BB_y^{-1}A_y T_\ell^{-1} \mid BB_y^{-1}\right]\begin{bmatrix} y_d \\ y_d^{(r)}\end{bmatrix}$$

$$A_{inv} \qquad\qquad B_{inv}$$

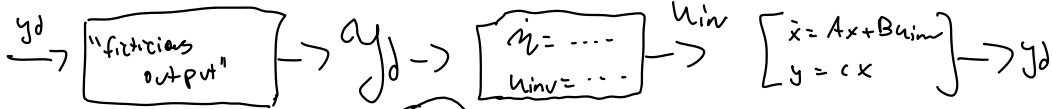
$$\boxed{\dot\eta = A_{inv}\eta + B_{inv}\, y_d}$$

Step 4: Write the Internal Dynamics

Step 4: Write the Internal Dynamics

Step 4: Write the Internal Dynamics

Step 5: Complete the Inverse System



$$A_{inv} = T_\eta \left[A - (BB_y^{-1}A_y) \right] T_r^{-1}$$

$$B_{inv} = \left[T_\eta \left[A - (BB_y^{-1}A_y) \right] T_l^{-1} \;\vdots\; T_\eta B B_y^{-1} \right]$$

$$C_{inv} = -B_y^{-1} A_y T_r^{-1}$$

$$D_{inv} = \left[-B_y^{-1} A_y T_l^{-1} \;\vdots\; B_y^{-1} \right]$$

$$\mathcal{Y}_d(t) = \begin{bmatrix} \xi_d(t) \\ y_d^{(r)}(t) \end{bmatrix}$$

$$\dot{\eta}_{ref} = A_{inv}\eta_{ref} + B_{inv}\mathcal{Y}_d$$

$$u_{inv} = C_{inv}\eta_{ref} + D_{inv}\mathcal{Y}_d$$

A_{inv}

- eigenvalues of A_{inv} are the zeros of orginal system
- $\Rightarrow$ A_{inv} unstable for non-min phase systems

Step 5.5: Solve the Inverse System (min phase system)

(a) $\dot{\eta} = A_{inv}\eta + B_{inv}y_d$

$u_{inv} = C_{inv}\eta + D_{inv}y_d$

} ODE -> solve, need $\eta(t_0)$

-> assume the system starts at equilibrium, at t_0

$\dot{x} = 0 \Rightarrow \dot{\eta} = 0 = A_{inv}\eta(t_0) + B_{inv}y_d(t_0)$

$\eta(t_0) = -A_{inv}^{-1}B_{in}y_d(t_0)$

-> simulate (b)

Step 6: For non-minimum phase systems

Step 6: For non-minimum phase systems

Step 6: For non-minimum phase systems

Matlab Example

Non-Minimum Phase System

$$\begin{aligned} \dot{x} &= Ax + Bu \\ y &= Cx \end{aligned} \longrightarrow \begin{aligned} \dot{\eta}_{ref} &= A_{inv}\eta_{ref} + B_{inv}\mathcal{Y}_d \\ u_{inv} &= C_{inv}\eta_{ref} + D_{inv}\mathcal{Y}_d \end{aligned} \longrightarrow \begin{aligned} \dot{\eta}_s &= A_s\eta_s + B_s\mathcal{Y}_d \\ \dot{\eta}_d &= A_u\eta_u + B_u\mathcal{Y}_d \end{aligned}$$

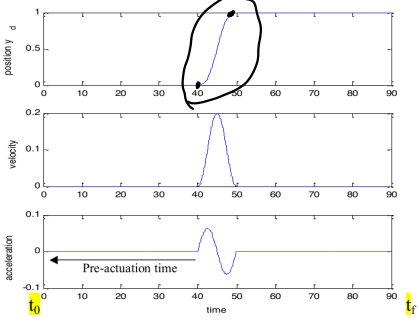
Focus: Internal Dynamics

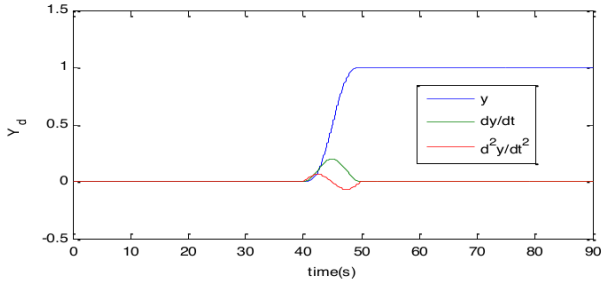
$$\dot{\eta}_s = A_s \eta_s + B_s \mathcal{Y}_d$$
$$\dot{\eta}_d = A_u \eta_u + B_u \mathcal{Y}_d$$

$$A_s = -0.95$$
$$A_u = 1.05$$

Desired Trajectory

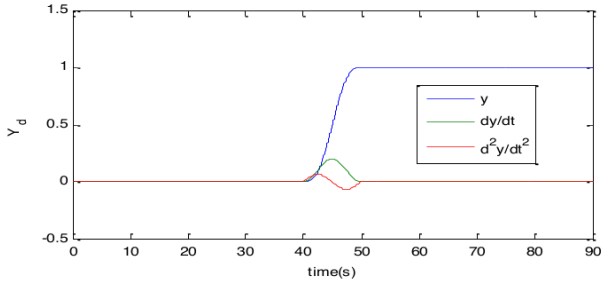
run it thru low pass filter





Solve Stable Subsystem

$$\dot{\eta}_s = A_s \eta_s + B_s \mathcal{Y}_d$$
$$Y_s = C_s \eta_s + D_s \mathcal{Y}_d$$
$$C_s = \begin{bmatrix} 1 \end{bmatrix}, \; D_s = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$$
$$\mathcal{Y}_d = \begin{bmatrix} y & \dot{y} & \ddot{y} \end{bmatrix}^T$$



Solve Stable Subsystem

$$\dot{\eta}_s = A_s\eta_s + B_s\mathcal{Y}_d$$
$$Y_s = C_s\eta_s + D_s\mathcal{Y}_d$$
$$C_s = \begin{bmatrix}1\end{bmatrix},\ D_s = \begin{bmatrix}0 & 0 & 0\end{bmatrix}$$
$$\mathcal{Y}_d = \begin{bmatrix}y & \dot{y} & \ddot{y}\end{bmatrix}^T$$

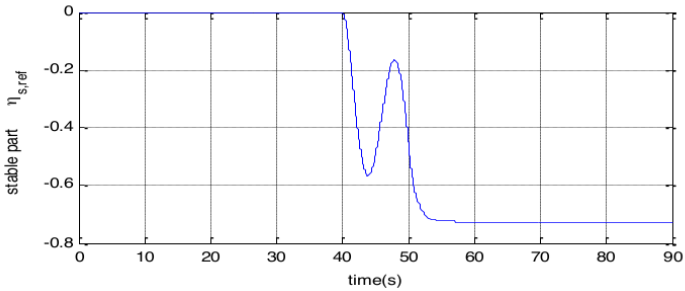
Initial condition (from equilibrium:)

$$0 = A_s\eta_s + B_s\mathcal{Y}_d$$
$$\eta_s(t_0) = -A_s^{-1}B_s\mathcal{Y}_d(t_0)$$

`sys_s=ss(`A_s`,` B_s`,` C_s`,` D_s`)`

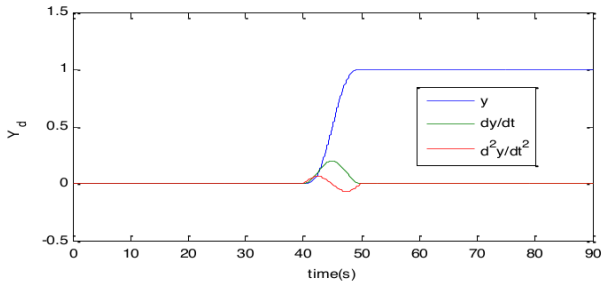
`[`$\mathcal{Y}_d$`, t,` $\eta_{s,ref}$`]=lsim(sys_s,` $\mathcal{Y}_d$`, t,` $\eta_s(t_0)$`)`

Solve Stable Subsystem



Solve Unstable Subsystem

$$\dot{\eta}_u = A_u \eta_u + B_u \mathcal{Y}_d$$
$$Y_u = C_u \eta_u + D_u \mathcal{Y}_d$$
$$C_u = \begin{bmatrix} 1 \end{bmatrix}, \; D_u = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$$
$$\mathcal{Y}_d = \begin{bmatrix} y & \dot{y} & \ddot{y} \end{bmatrix}^T$$



Solve Unstable Subsystem

$$\dot{\eta}_u = A_u \eta_u + B_u \mathcal{Y}_d$$
$$Y_u = C_u \eta_u + D_u \mathcal{Y}_d$$
$$C_u = \begin{bmatrix} 1 \end{bmatrix}, \; D_u = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$$
$$\mathcal{Y}_d = \begin{bmatrix} y & \dot{y} & \ddot{y} \end{bmatrix}^T$$

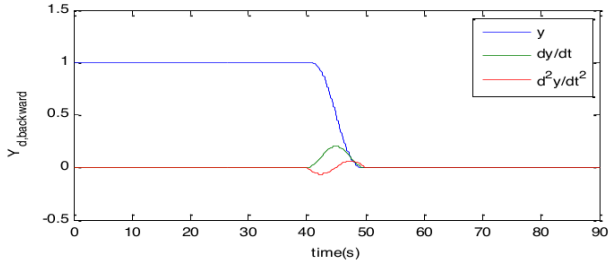
Solve it backwards in time:

$$\dot{\eta}_u = -A_u \eta_u + -B_u \mathcal{Y}_d$$
$$Y_u = C_u \eta_u + D_u \mathcal{Y}_d$$

Solve Unstable Subsystem

$$\dot{\eta}_u = A_u \eta_u + B_u \mathcal{Y}_d$$
$$Y_u = C_u \eta_u + D_u \mathcal{Y}_d$$
$$C_u = \begin{bmatrix} 1 \end{bmatrix}, \; D_u = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$$
$$\mathcal{Y}_d = \begin{bmatrix} y & \dot{y} & \ddot{y} \end{bmatrix}^T$$

Solve it backwards in time:



Solve Unstable Subsystem

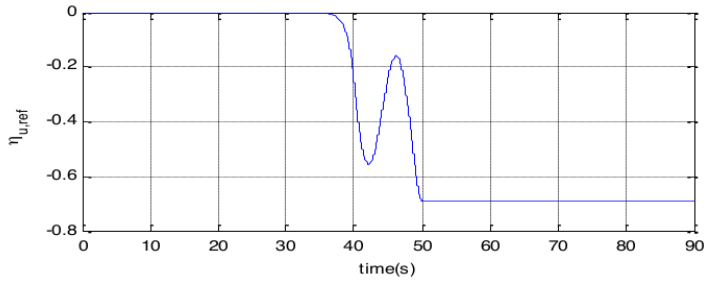
$$\dot{\eta}_u = -A_u \eta_u + -B_u \mathcal{Y}_d$$
$$Y_u = C_u \eta_u + D_u \mathcal{Y}_d$$

Initial condition (from equilibrium:)

$$0 = A_u \eta_u + B_u \mathcal{Y}_d$$
$$\eta_u(t_f) = -A_u^{-1} B_u \mathcal{Y}_d(t_f)$$

`sys_u_bw = ss(`$-A_u$`, `$-B_u$`, `C_u`, `D_u`)`
$\mathcal{Y}_{d,bw}$ `= fliplr(`$\mathcal{Y}_d$`)`
`[`$\mathcal{Y}_{ub}$`, t, `η_{ub}`] = lsim(sys_u_bw, `$\mathcal{Y}_{d,bw}$`, `$\eta_u(t_f)$`)`
$\eta_{u,ref}$ `= fliplr(`η_u^T`)`

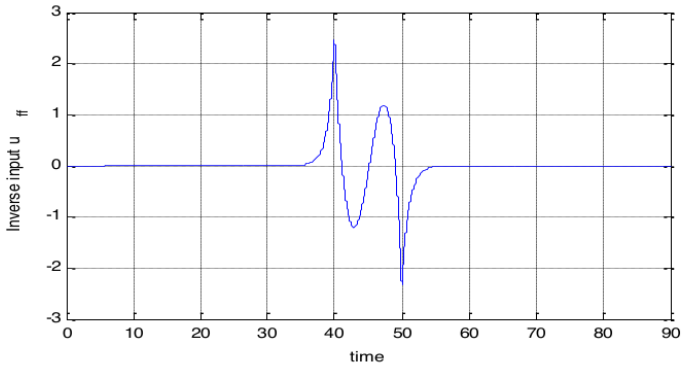
Solve Stable Subsystem



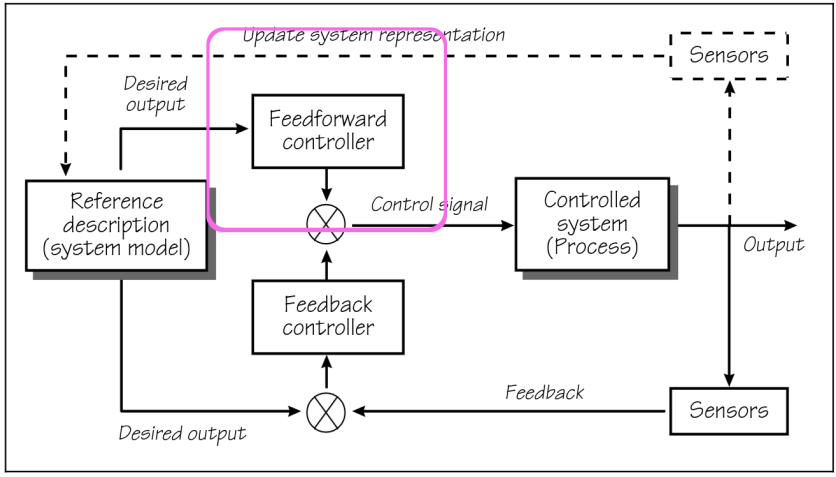
Find Reference Input

$$\eta_{ref} = T_{split}\begin{bmatrix}\eta_{s,ref}\\ \eta_{u,ref}\end{bmatrix}$$

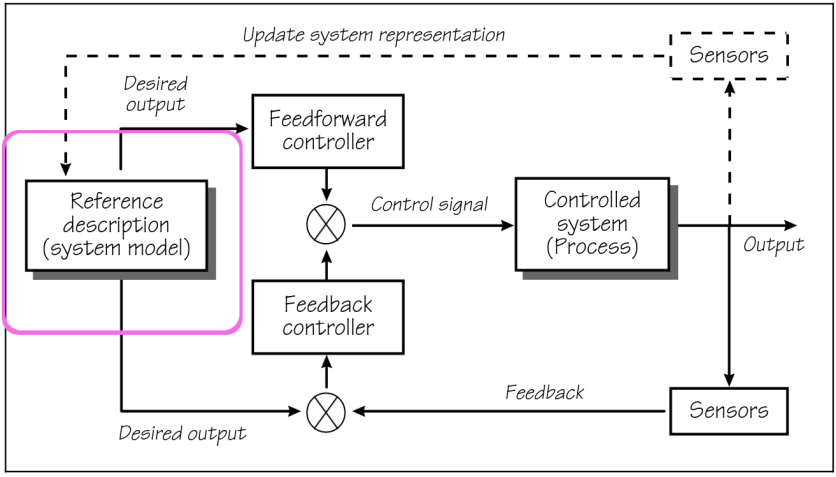
$$u_{inv}(t) = C_{inv}\eta_{ref} + D_{inv}\mathcal{Y}_d$$



Today


Update system representation
Sensors
Desired output
Feedforward controller
Reference description (system model)
Control signal
Controlled system (Process)
Output
Feedback controller
Feedback
Sensors
Desired output

Next Time


Update system representation
Sensors
Desired output
Feedforward controller
Reference description (system model)
Control signal
Controlled system (Process)
Output
Feedback controller
Feedback
Sensors
Desired output